Report: Assignment 3 - Bridge Pattern

Notification & Branding Service

by Abraimov Yeskendir, SE-2502

1. Introduction

For this assignment I took my Notification & Branding Service from Assignment 2 and refactored it with the Bridge pattern (Option B, topic "Notification × delivery channel").

The service already had two dimensions that change independently: the **brand** that decides how a message looks (Corporate or Marketing/Promo) and the **channel** that delivers it (Email, SMS, Push). In Assignment 2 they were connected badly. The sender was bound to its channel by inheritance (`EmailNotificationSender`, `SmsNotificationSender`, `PushNotificationSender`), and the brand was applied by hand in `main` before calling `dispatch()`. If I wanted the sender to also know its brand, inheritance would give me a class for every pair - `CorporateEmailSender`, `PromoSmsSender`, and so on. The number of classes grows as brands × channels.

That is exactly the problem Bridge solves. I made `NotificationSender` the Abstraction: it holds a reference to a `Notification` (the channel) and delegates the delivery to it. The brand-specific senders became Refined Abstractions, and the channels became Concrete Implementors. Now 2 brands × 3 channels = 6 combinations need only 2 + 3 classes, and the same sender object can switch its channel at runtime with `setNotification()`.

The Abstract Factory part from Assignment 2 was kept without changes - the refined abstractions use it to build the styled text. Lecture 4 mentions that Abstract Factory and Bridge can be used together, and in this project it came out naturally.

2. UML Class Diagram

The left side is the Abstraction hierarchy (`NotificationSender` and its two brand senders), the right side is the Implementor hierarchy (`Notification` and its three channels). The aggregation arrow between `NotificationSender` and `Notification` is the "bridge" - the only link between the two sides.

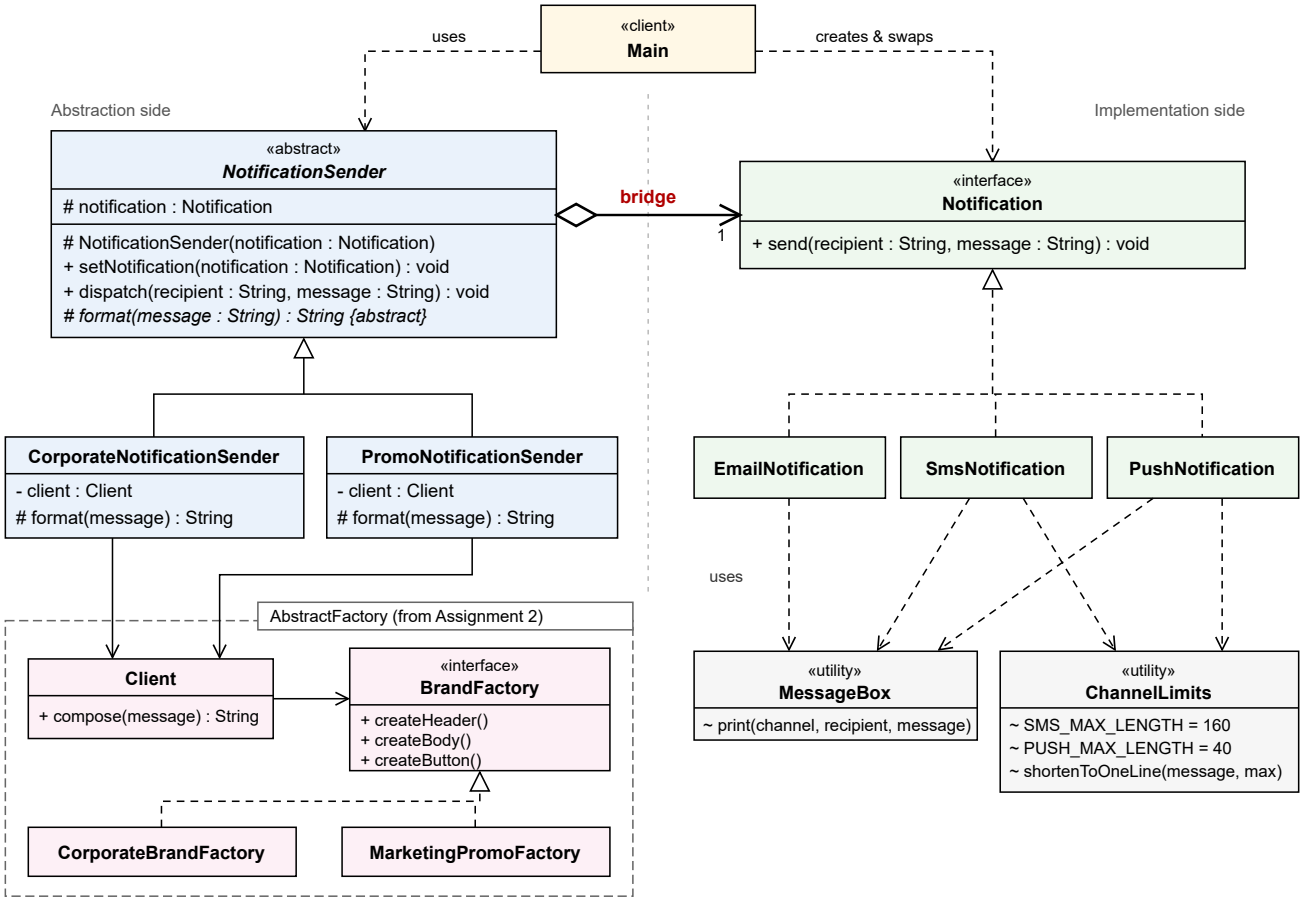


Figure 1 - Bridge between NotificationSender (Abstraction) and Notification (Implementor)

Role	Class
Abstraction	`NotificationSender`
Refined Abstraction	`CorporateNotificationSender`, `PromoNotificationSender`
Implementor	`Notification`
Concrete Implementor	`EmailNotification`, `SmsNotification`, `PushNotification`
Client	`Main`

3. Clean Code Principles Applied

1. Separation of Abstraction and Implementation Responsibilities

Clean Code, Ch. 10 says a class should have only one reason to change. In this project the sender side only decides **how the message looks** (the brand), and the channel side only decides **how it is delivered** - email shows everything, SMS makes one line of max 160 characters, push shows a 40-character preview. The channel doesn't know that brands exist, and the sender doesn't know which channel it has. In Assignment 2 `NotificationSender` mixed everything: it printed the box, it chose the channel through inheritance, and it created the `Notification` but never actually called `send()`.

Before (Assignment 2, NotificationSender.java):

```
public void dispatch(String recipient, String message) {
    String channel = getChannelName();

    System.out.println("┌─ " + channel + " " + "─".repeat(30));
    System.out.println("│ To: " + recipient);
    ...
    System.out.println("└" + "─".repeat(33));

    Notification notification = createNotification();

    System.out.println();
}
```

After - NotificationSender.java (only delegates through the bridge):

```
public void dispatch(String recipient, String message) {
    notification.send(recipient, format(message));
}

//sender decides how the message looks, channel decides how it is delivered
protected abstract String format(String message);
```

The client never calls `send()` on a channel directly - it only calls `dispatch()` on the sender, so implementor details don't leak to the client.

2. Meaningful Names Distinguishing the Roles

Clean Code, Ch. 2 says names should reveal intent. The Abstraction side is named after what it does - it *sends*: `NotificationSender`, `CorporateNotificationSender`, `PromoNotificationSender`. The Implementor side is named after the channel: `EmailNotification`, `SmsNotification`, `PushNotification`. The setter is called `setNotification()`, so it says exactly which part gets replaced. I also renamed the entry point from `main` to `Main`, because a Java class name should start with a capital letter and shouldn't look like the method inside it.

Before (Assignment 2, main.java):

```
public class main {
    public static void main(String[] args) { ... }
}
```

After - Main.java:

```
public class Main {
    public static void main(String[] args) { ... }
}
```

3. Small, Focused Classes on Both Sides

Each class has one job. A refined abstraction only knows its brand, and a channel only knows its own delivery rules. Limits are not hard-coded inside the channels - they are named constants in `ChannelLimits`, the same way I used `CarSeats` in Assignment 1 to avoid magic numbers.

PromoNotificationSender.java - the whole Refined Abstraction:

```
public class PromoNotificationSender extends NotificationSender {
    //Refined Abstraction - styles message with promo brand from Abstract Factory
    private final Client client = new Client(new MarketingPromoFactory());

    public PromoNotificationSender(Notification notification){
        super(notification);
    }

    @Override
    protected String format(String message){
        return client.compose(message);
    }
}
```

ChannelLimits.java - no magic numbers in channels:

```
static final int SMS_MAX_LENGTH = 160;
static final int PUSH_MAX_LENGTH = 40;
```

4. No Duplicated Logic Between Concrete Implementors

Martin treats duplication as one of the main things to remove from code, because the same logic in two places means two places to update. All three channels print the same box, and SMS and Push both have to make the message one line and cut it. In Assignment 2 the box was inside the abstract creator. In Bridge the Implementor is an interface, so I moved the shared parts into two small package-private helpers, `MessageBox` and `ChannelLimits`, instead of copying them into every channel.

Before - the same code would be copied into each channel:

```
public class SmsNotification implements Notification {
    public void send(String recipient, String message) {
        System.out.println("┌─ SMS " + "─".repeat(30));
        System.out.println("│ To: " + recipient);
        String text = message.replace("\n", " ");
        if (text.length() > 160) {
            text = text.substring(0, 157) + "...";
        }
        System.out.println("│ " + text);
        System.out.println("└" + "─".repeat(33));
    }
}
// and almost the same in EmailNotification and PushNotification
```

After - SmsNotification.java:

```
@Override
public void send(String recipient, String message) {
    String text = ChannelLimits.shortenToOneLine(message, ChannelLimits.SMS_MAX_LENGTH);
    MessageBox.print("SMS", recipient, text);
}
```

5. Backward-Compatible Design (Open/Closed)

Adding a new Concrete Implementor requires no change to the Abstraction. A Telegram channel would be one new class, and `NotificationSender`, `CorporateNotificationSender` and `PromoNotificationSender` would stay untouched. In Assignment 2 the same thing needed two classes (a product and a creator). It also works the other way - a new brand is one new sender class, and no channel has to change.

A new channel is just one class:

```
public class TelegramNotification implements Notification {
    @Override
    public void send(String recipient, String message) {
        MessageBox.print("TELEGRAM", recipient, message);
    }
}
```

Main.java - existing senders can use it right away:

```
promoSender.setNotification(new TelegramNotification());
promoSender.dispatch("Yeska", "Hello");
```

4. Conclusion

Refactoring my own Assignment 2 made the benefit of Bridge very clear. Before, the channel was fixed by the sender's class, and the brand was glued on by hand in `main`. Now 2 brands and 3 channels give 6 combinations with only 2 + 3 classes, and with 5 brands and 5 channels it would be 10 classes instead of 25. Switching the channel at runtime with one `setNotification()` call is something inheritance can't do at all - an object can't change its superclass after it is created. In the client this is shown both in code and in a live demo, where the user types `email`, `sms` or `push` and the same sender re-sends the message.

The trade-off is extra complexity: one more level of delegation, so following `dispatch()` means jumping from the sender to the channel. With only one brand and one channel this would be over-engineering. I also noticed that the Implementor interface has to be designed carefully - `send(recipient, message)` works for all current channels, but if a future channel needed something more (for example attachments), the interface and every channel would have to change.

Compared to Factory Method from Assignment 2: Factory Method only decided which channel object to create, and that choice was fixed by the subclass. Bridge keeps two separate hierarchies and lets them be combined freely, even at runtime. Compared to Adapter: Adapter is used afterwards to make an existing incompatible interface fit, while Bridge is planned so that both sides can grow independently.

5. GitHub Repository

Repository: https://github.com/Naitskaut/SDP
Assignment 3 folder: https://github.com/Naitskaut/SDP/tree/main/Assignment%203